

Formal Languages and Compilers

(Linguaggi Formali e Compilatori)

prof. Luca Breveglieri

(prof. S. Crespi Reghizzi, prof. A. Morzenti)

Written exam - 28 June 2013 - Part I: Theory

WARNING - THE SYNTAX ANALYSIS EXERCISES MUST BE SOLVED BY THE ELR / ELL THEORY

NAME:

SURNAME:

ID:

SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam consists of two parts:
 - I (80%) Theory:
 1. regular expressions and finite automata
 2. free grammars and pushdown automata
 3. syntax analysis and parsing
 4. translation and semantic analysis
 - II (20%) Practice on Flex and Bison
- To pass the exam, the candidate must succeed in both parts (I and II), in one call or more calls separately, but within one year.
- To pass part I (theory) one must answer the mandatory (not optional) questions. Notice the full grade is achieved by answering the optional questions.
- The exam is open book (texts and personal notes are admitted).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets nor replace the existing ones.
- Time: Part I (theory): 2h.15m - Part II (practice): 60m

1 Regular Expressions and Finite Automata 20%

1. Consider the two regular expressions X and Y below:

$$X = (a^+ b)^* \qquad Y = (a b)^+ b^+$$

Answer the following questions:

- (a) As simply as possible, build a finite automaton A (not necessarily deterministic) that recognizes the union language $L(X) \cup L(Y)$.
 - (b) Again for the union language $L(X) \cup L(Y)$, find a deterministic automaton A' by a method at choice.
 - (c) For the concatenation language $L(X) \cdot L(Y)$, write a regular expression Z , verify that it is not ambiguous, and if it is not so, then modify it so as to remove its ambiguity.
 - (d) (optional) Verify that the deterministic automaton A' found at point (b) is minimal, and if it is not so, minimize it.
-

2 Free Grammars and Pushdown Automata 20%

1. Consider a subset of the Dyck language with three parenthesis types (round, square and curly), defined through the following clauses:
 - (a) in the sequences of the parenthesis pairs (nests) at the same level, a pair with round parentheses at the outer level can be followed only by a pair with square parentheses at the *outer* level, a square pair only by a curly one, and a curly pair only by a round one
 - (b) the sequence of parenthesis pairs at the *outermost* level (those not enclosed in any other parentheses) starts by a pair with round parentheses at the outer level
 - (c) a sequence of parenthesis pairs *immediately* enclosed in a pair of round parentheses (square and curly, respectively), starts by a pair with square parentheses at the outer level (curly and round, respectively)

Sample strings *belonging* to the language:

$$() [] \{ \} () \quad ([]) [\{ \} (())] \{ () [\{ \}] \} \quad () [] \{ \} \left([\{ (()) \}] \right) []$$

Sample strings *not belonging* to the language (the leftmost violation found in reading the string is underlined and briefly described, but others may follow):

<i>erroneous string</i>	<i>leftmost clause violated and description</i>
$(\underline{() \{ \} }) [] ()$	violates (a): round not followed by square; more violations of (a) follow
$[\underline{[] \{ \} }] [] \{ ([\{ \}]) \} ()$	violates (b): at the outermost level the string does not start by round; more violations of (a) follow
$([]) [\{ \} (\underline{\{ \} })] \{ [] [\{ \}] \}$	violates (c): pair immediately enclosed in round does not start by square; more violations of (a) and (b) follow

Answer the following questions:

- (a) Write a grammar that generates the language described above.
- (b) (optional) Draw the syntax tree for the valid sample string below:

$$([]) [\{ \} (())] \{ () [] \}$$

2. Consider part of the syntax of the *HTML* language for hypertexts. Here is how a draft *HTML* document may look like:

```
<html xmlns="www.w3.org/1999/xhtml" lang="en">
  <head>
    <title>FDTC 2013</title>
  </head>
  <body>
    <div id="main">
      <h1>Location</h1>
      <a href="/shared/UCSB_map1.png">
        
        
      </a>
    </div>
  </body>
</html>
```

Basically the document is an aggregate of records. Each record has an opening mark `<tag ... >` and a closing one `</tag>` with identical *tags* (or labels), and may contain other records between these two marks. Overall it is a typical block structure.

The document is exactly one record `html`, which must contain exactly one record `head` followed by one record `body`, and these may contain others freely concatenated and nested, tagged in any way; the records `head` and `body` may not be empty.

The specifications below sufficiently clarify the details of the syntax hinted above:

- A record has the following general structure:

```
<tag field_1 ... field_n> ... nested records ... </tag>
```

If a record does not have any nested records, it can be (optionally) denoted as:

```
<tag field_1 ... field_n /tag>
```

```
<tag field_1 ... field_n />
```

There may be one or more fields `field_i` ($n \geq 1$), or they may miss ($n = 0$).

- The fields `field_i` have the following structure, with a name and a value:

```
field_name = 'string'
```

```
field_name = number
```

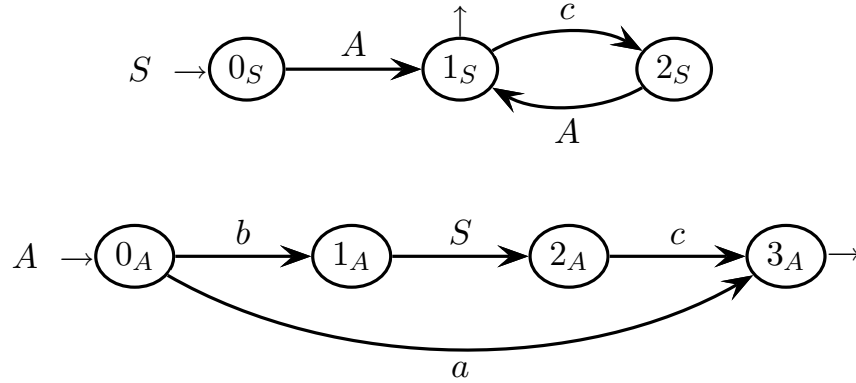
where `string` and `number` are an alphanumerical string and a decimal integer.

- Suppose that the terminals `tag`, `name`, `string` and `number` are a record tag, a field name, a string and an integer, respectively, with no need of further expansion.

Write an extended grammar (*EBNF*), not ambiguous, and if necessary, reasonably complete the possibly undefined aspects of the description.

3 Syntax Analysis and Parsing 20%

1. Consider the extended grammar (*EBNF*) that corresponds to the network of finite recursive machines shown in the figure below (axiom S):



Answer the following questions, and reasonably explain why:

- (a) Determine if the grammar is *ELL*(1), by directly verifying on the *Parser Control Flow Graph* (*PCFG*), without passing through the pilot.
- (b) Determine if the grammar is *ELR*(1), by building the pilot.
- (c) (optional) Determine if by scaling up to a prospection $k = 2$, the grammar is deterministic *ELL*(2).

CAUTION: read the warning on the cover page.

4 Translation and Semantic Analysis 20%

1. One wishes one transformed a program written in C or Java style, which uses curly brackets as delimiters, into an equivalent one in Pascal or Ada style, which uses the keywords *begin*, *end*, *then*, *else* and *endif*. The source and image languages, and how to transform the former into the latter, are illustrated by the example below:

<pre> { if c₁ { s₁; s₂; s₃; }; else { s₄; s₅; if c₂ { s₆; }; s₇; }; while c₃ do { s₈; s₉; }; }; </pre>	<pre> begin if c₁ then s₁; s₂; s₃ else s₄; s₅; if c₂ then s₆ endif s₇ endif while c₃ do begin s₈; s₉ end end </pre>
--	---

The source and image languages are defined by the *EBNF* grammars G_1 and G_2 below, which must be changed into not extended (*BNF*) to write the transduction.

$$\begin{aligned}
 G_1 & \left\{ \begin{array}{l} \langle \text{prog1} \rangle \rightarrow \{ ' (\langle \text{stat1} \rangle ;)^+ ' \}; \\ \langle \text{stat1} \rangle \rightarrow \text{if } c \{ ' (\langle \text{stat1} \rangle ;)^+ ' \} [; \text{else } \{ ' (\langle \text{stat1} \rangle ;)^+ ' \}] \\ \langle \text{stat1} \rangle \rightarrow \text{while } c \text{ do } \{ ' (\langle \text{stat1} \rangle ;)^+ ' \} \\ \langle \text{stat1} \rangle \rightarrow s \quad \text{-- any simple statement} \end{array} \right. \\
 G_2 & \left\{ \begin{array}{l} \langle \text{prog2} \rangle \rightarrow \text{begin } (\langle \text{stat2} \rangle ;)^* \langle \text{stat2} \rangle \text{ end} \\ \langle \text{stat2} \rangle \rightarrow \text{if } c \text{ then } (\langle \text{stat2} \rangle ;)^* \langle \text{stat2} \rangle [\text{else } (\langle \text{stat2} \rangle ;)^* \langle \text{stat2} \rangle] \text{ endif} \\ \langle \text{stat2} \rangle \rightarrow \text{while } c \text{ do begin } (\langle \text{stat2} \rangle ;)^* \langle \text{stat2} \rangle \text{ end} \\ \langle \text{stat2} \rangle \rightarrow s \quad \text{-- any simple statement} \end{array} \right.
 \end{aligned}$$

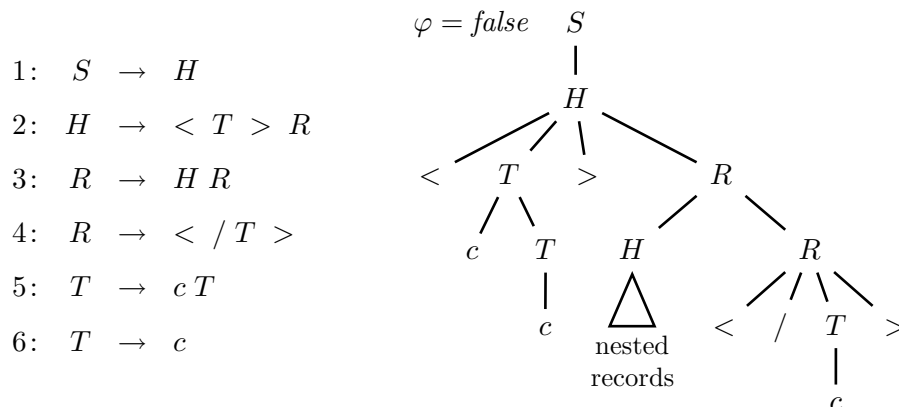
The axioms of G_1 and G_2 are *prog1* and *prog2*. Caution: the semicolon “;” is missing in the image language, if it is immediately followed by a keyword *end* or *endif*.

Answer the following questions:

- (a) Write a syntax transduction scheme (no attributes) for the translation above.
- (b) (optional) Draw the translation syntax tree of the sample source string below:

$$\{ \text{if } c_1 \{ s_1; \}; \text{ else } \{ \text{if } c_2 \{ \text{while } c_3 \text{ do } \{ s_2; \}; \}; s_3; \}; \};$$

2. Consider the syntax below, in the not extended form (*BNF*), which generates a part (very simplified) of the *HTML* language for hypertexts (axiom S).



Aside the syntax support, there is a sample syntax tree (partial). Those who want so, may see the text of ex. 2.2 for the few notions about *HTML* that suffice for here.

The syntax above does not ensure that the opening $< T >$ and closing $< / T >$ tags of a record match. One wishes one added such a control by means of attributes.

Answer the following questions:

- (a) Write the semantic functions to compute a boolean attribute φ in the tree root S , which is *true* if and only if the opening and closing tags of each record match (see the sample tree above). Use the attributes listed below (do not add more attributes), and specify the type of the *tag* and *carry* attributes:

<i>name</i>	<i>type</i>	<i>symbol</i>	<i>domain</i>	<i>meaning</i>
φ	left	H, R, S	boolean	<i>true</i> iff the opening and closing tags of each record coincide
<i>tag</i>		T	string	opening or closing tag
<i>carry</i>		R	string	tag transported through the tree for comparing it with the corresponding tag

- (b) Discuss if the attribute grammar found at point (a), is one sweep. Since the syntax is $LL(2)$, discuss if the semantic analyzer based on such a grammar can be integrated with the syntactic $LL(2)$ one.
- (c) (optional) One wishes the semantic analyzer emitted an error message as soon as it has finished to read the closing tag of a record and finds out the tag does not match the opening one (of the same record). Can it be done and integrated with the syntactic $LL(2)$ analyzer? (explain why) If it is impossible, argue how to change the semantic functions (not the syntax) so as to make it possible.

syntax

semantic - question (a)

1: $S_0 \rightarrow H_1$

2: $H_0 \rightarrow < T_1 > R_2$

3: $R_0 \rightarrow H_1 R_2$

4: $R_0 \rightarrow < / T_1 >$

5: $T_0 \rightarrow c T_1$

6: $T_0 \rightarrow c$

syntax

semantic - question (c)

1: $S_0 \rightarrow H_1$

2: $H_0 \rightarrow < T_1 > R_2$

3: $R_0 \rightarrow H_1 R_2$

4: $R_0 \rightarrow < / T_1 >$

5: $T_0 \rightarrow c T_1$

6: $T_0 \rightarrow c$

